

LSV 2026 Programming Assignment 1

Report

Wei-Hung Chan, R15921041

September 19, 2026

1 Exercise 2: Using ABC

1.1 2(a) Two-bit unsigned multiplier BLIF

The file `lsv/pa1/mul.blif` implements $Y = A \times B$ where $A = (a_1a_0)$, $B = (b_1b_0)$, and $Y = (y_3y_2y_1y_0)$. Partial products are formed first, then the product bits:

```
p00 = a0 & b0;    p01 = a0 & b1;
p10 = a1 & b0;    p11 = a1 & b1;
y0 = p00;
y1 = p01 XOR p10;  c1 = p01 AND p10;
y2 = p11 XOR c1;   y3 = p11 AND c1;
```

All 4×4 input combinations were checked; the BLIF matches $a \times b$.

1.2 2(b) ABC command walkthrough

Commands run on `mul.blif`:

```
read lsv/pa1/mul.blif
print_stats
show
strash
show
collapse
show_bdd -g
```

print_stats after read:

```
mul : i/o = 4/4  lat = 0  nd = 9  edge = 17  cube = 11  lev = 3
```

print_stats after strash:

```
mul : i/o = 4/4  lat = 0  and = 12  lev = 4
```

Figures below are produced by ABC's `show/show_bdd` (PostScript rendered to PNG; the machine has no GUI display for `gv`).

Network structure visualized by ABC
 Benchmark "mul". Time was Sat Sep 19 15:08:13 2026.

The network contains 9 logic nodes and 0 latches.

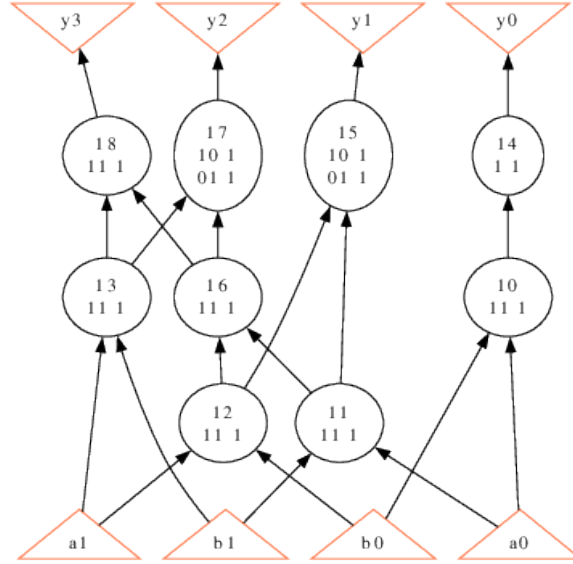


Figure 1: Network after read (show).

Network structure visualized by ABC
 Benchmark "mul". Time was Sat Sep 19 15:08:14 2026.

The network contains 12 logic nodes and 0 latches.

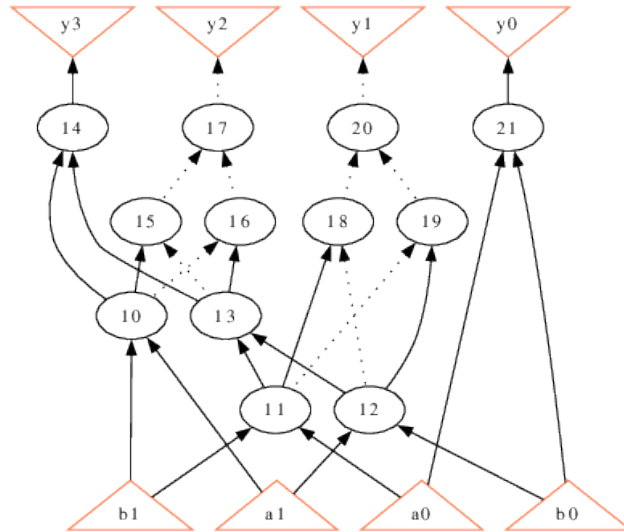


Figure 2: Structurally hashed AIG after strash (show).

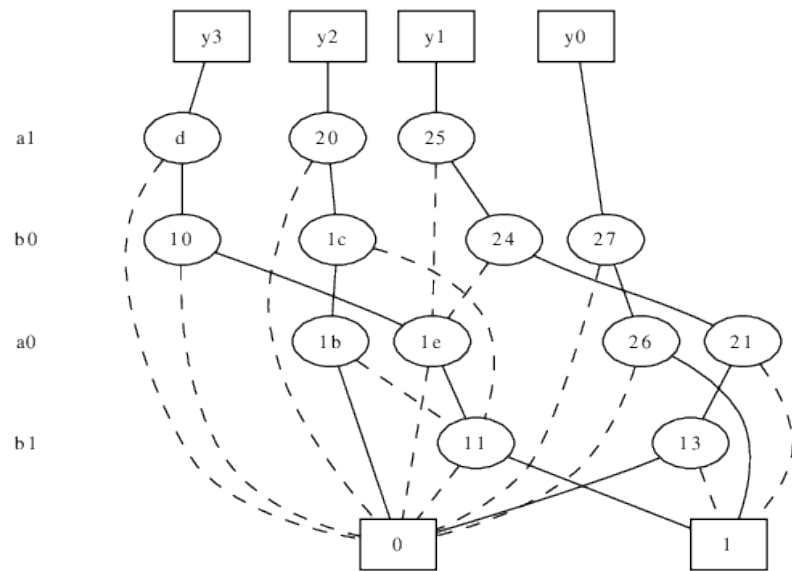


Figure 3: Collapsed global BDDs after `collapse (show_bdd -g)`.

2 Exercise 3: ABC Boolean Representations

2.1 3(a) Comparisons

Logic network in AIG (aig) vs. structurally hashed AIG (strash).

```
# after aig:
mul : i/o = 4/4   nd = 9   edge = 17   aig = 12   lev = 3
# after strash:
mul : i/o = 4/4   and = 12   lev = 4
```

Findings: **aig** keeps the original multi-level network topology and stores each node's local function as a (possibly multi-gate) AIG fragment (**nd=9**, **aig=12**). **strash** flattens everything into one globally hashed AND-inverter graph (**and=12**). Structural hashing merges identical gates, so the representation is unique and level count can change.

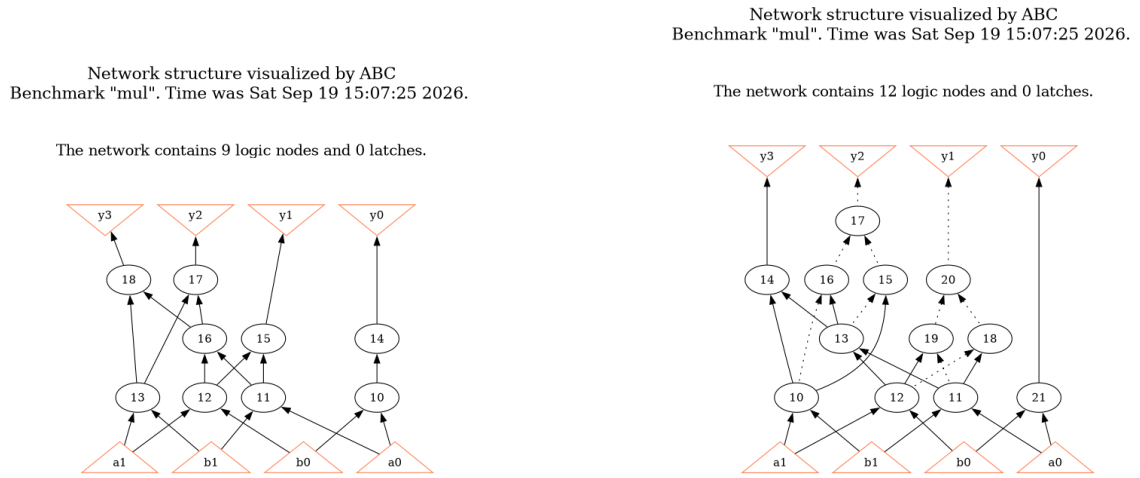


Figure 4: Left: after aig. Right: after strash.

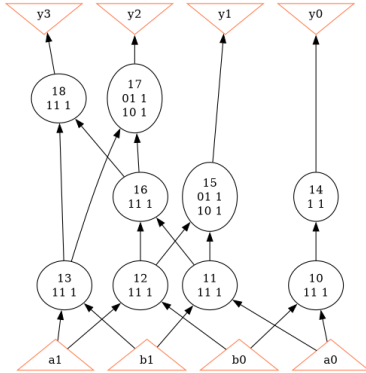
Logic network in BDD (bdd) vs. collapsed BDD (collapse).

```
# after bdd:
mul : i/o = 4/4   nd = 9   edge = 17   bdd = 16   lev = 3
# after collapse:
mul : i/o = 4/4   nd = 4   edge = 14   bdd = 14   lev = 1
```

Findings: **bdd** builds a local BDD for each internal node while preserving the multi-level structure (**nd=9**, **lev=3**). **collapse** builds global BDDs for the primary outputs only, eliminating internal nodes (**nd=4** = four POs, **lev=1**).

Network structure visualized by ABC
Benchmark "mul". Time was Sat Sep 19 15:07:25 2026.

The network contains 9 logic nodes and 0 latches.



Network structure visualized by ABC
Benchmark "mul". Time was Sat Sep 19 15:07:25 2026.

The network contains 4 logic nodes and 0 latches.

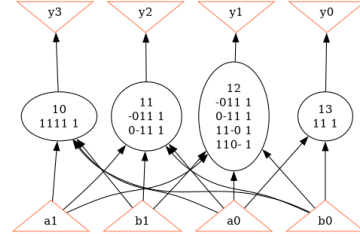


Figure 5: Left: after bdd. Right: after collapse.

2.2 3(b) Strashed AIG → SOP logic network

Command sequence:

```
read lsv/pa1/mul.blif
strash
logic
sop
print_stats
```

Result:

```
mul : i/o = 4/4   nd = 12   edge = 24   cube = 12   lev = 4
```

`logic` converts the structural AIG into a logic network; `sop` expresses every node function in sum-of-products form (`cube` appears in `print_stats`). Verified with `lsv_print_nodes`, which prints each node's SOP cubes.

3 Exercise 4: Programming ABC (summary)

Commands implemented under `src/ext-lsv/`:

- `lsv_cut_tt <k>` — irredundant k -feasible cuts + truth tables
- `lsv_cut_bddsize <k>` — irredundant k -feasible cuts + ROBDD sizes

Verified against the handout Fig. 1 example (`lsv/pa1/example.blif`, $k = 3$); outputs match exactly. Dominated cuts are omitted (TA clarification on issue #983).